

Week06 | 实验 | 把 Week3—Week5 链路组织成 Dagster 数据工厂，并完成一次分区回填 + 质量验证 + 运行证据交付

本页结构

跑出 Week06 的最小资产化数据工厂闭环	1
这次实验要完成什么	1
参考实验时间	2
本次实验产出	2
先看一张闭环图	2
实验不变量	2
Step 0 准备环境与运行边界	3
Step 1 验证 Week06 foundation 是否齐全	3
Step 2 验证 Definitions 可加载	4
Step 3 观察 source / manifest, 不要把观察误认为资产生成	5
Step 4 Materialize 一个 daily partition	6
Step 5 安全制造缺口	7
Step 6 生成并解读 backfill dry-run plan	7
Step 7 运行 5 类 asset checks	8
Step 8 生成 run evidence 并校验 schema	9
Step 9 按 runbook 完成交接记录	10
验收清单	11
常见错误与排查	11
延伸阅读	11

跑出 Week06 的最小资产化数据工厂闭环

这次实验不是做一个孤立 Dagster demo，而是把 Week03—Week05 已经形成的输入、表状态、指标口径组织成一个可运营的数据工厂。

你要完成的闭环是：

Definitions loadable -> asset graph -> daily partition -> dry-run backfill plan -> asset checks -> run evidence -> runbook 交接。

这次实验要完成什么

完成实验后，你应该能回答 6 个问题：

- Week06 的 Definitions 是否能被 runtime 正确加载？
- 最小 asset graph 里有哪些 asset key，哪些是 source / manifest / factory / evidence？
- 默认分区 2026-04-17 的输入、输出和缺口是什么？
- backfill dry-run plan 为什么建议当前动作，而不是直接重跑全量？
- 5 类 asset checks 分别给出了什么结果和 reason code？
- run evidence 能否让下游同学判断：继续、人工复核、hold，还是仅限 dry-run？

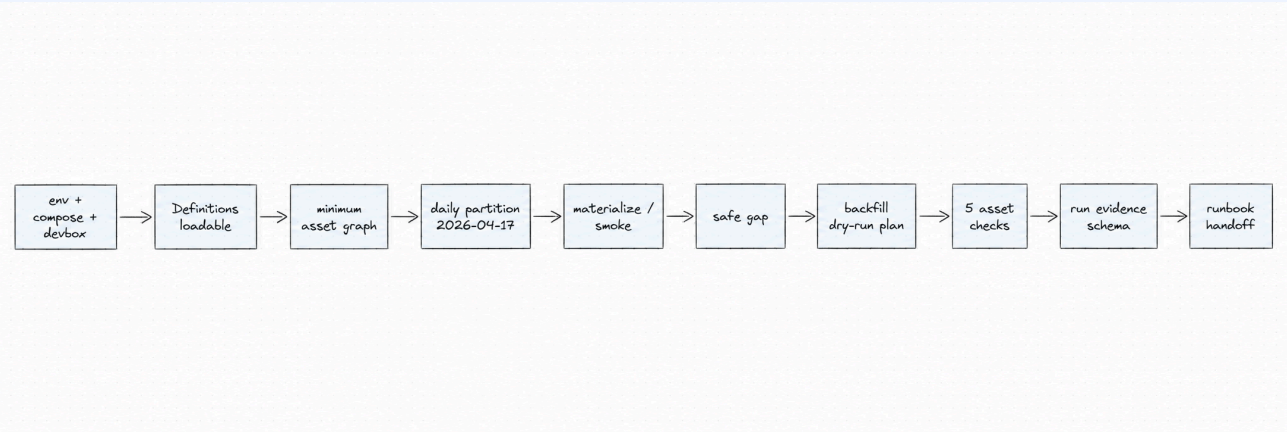
参考实验时间

按课程统一节奏完成即可。 建议预留 90—120 分钟：前半段跑通最小闭环，后半段整理 evidence 与 runbook 交接记录。

本次实验产出

类型	文件或目录	作用
Blueprint	docs/blueprints/ week06/week06-data-factory- blueprint.md	说明 Week06 数据工厂边界
Asset graph	docs/blueprints/week06/ week06-asset-graph.md	记录最小 asset key 与依赖
Partition strategy	docs/blueprints/week06/ week06-partition-backfill- strategy.md	说明分区、回填、重放策略
Evidence schema	contracts/run_evidence/week06_run_evidence_schema.json	约束 run evidence 结构
Backfill plan	reports/week06/backfill/ *.json	记录 dry-run 计划
Check results	reports/week06/checks/*.json	记录 asset checks 结果
Run evidence	reports/week06/run_evidence/ *.json	给下游消费的运行证据
Runbook	runbooks/week06- data-factory.md	给另一个人复现和接手
Lab summary	reports/week06/week06_lab_execution_summary.md	本次实验的最终说明

先看一张闭环图



实验不变量

这 6 条是本实验的最低验收线：

不变量	最低要求
Definitions 可加载	tests/integration/ test_week06_definitions_loadable.py 通过
Asset graph 可描述	能列出 source / factory / evidence 的最小 asset key

不变量	最低要求
默认分区可生成 dry-run plan	2026-04-17 有 backfill plan
5 类 checks 有结果	passed / warning / skipped / failed 都必须有 reason
Run evidence schema 通过	required 字段完整, optional 依赖不 fake passed
Runbook 可交接	另一个人能根据 summary 判断下游是否可继续

Step 0 | 准备环境与运行边界

为什么做这一步：

Week06 不是只跑一个 Python 脚本。你要确认 `.env.local`、Docker Compose、Postgres、MinIO、Dagster、devbox 使用的是同一套 runtime 边界，否则后续会出现“devbox 能跑、Dagster 看不到”的假通过。

执行命令：

```
# 可执行：已与项目 runbook 对齐
cp infra/env/.env.example infra/env/.env.local

docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml up -d --build

docker compose --env-file infra/env/.env.local -f infra/docker-compose.yml ps
```

期望看到什么：

- `infra/env/.env.local` 存在。
- Compose 服务启动完成。
- `postgres`、`minio`、Dagster 相关服务处于 `running` / `healthy` 或正在启动状态。

怎么看结果：

打开 `http://localhost:3000`，确认 Dagster UI 能访问。刚启动时短暂不可用可以等待 30—60 秒再刷新。

常见错误：

- `.env.local` 不存在：重新执行 `cp infra/env/.env.example infra/env/.env.local`。
- 端口冲突：确认 Docker / Podman 不要同时占用同一批端口。
- Dagster UI 打不开：先看 `docker compose ... ps`，不要直接改课程代码。

下一步进入条件：

Compose 服务能启动，且 `devbox` 命令可以运行：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox python --version
```

Step 1 | 验证 Week06 foundation 是否齐全

为什么做这一步：

先确认 Week06 的文档、schema、runbook、报告目录存在。否则后面即使跑出结果，也没有统一位置承接 evidence。

执行命令：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m json.tool contracts/run_evidence/week06_run_evidence.schema.json

docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    ls docs/blueprints/week06

docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    ls runbooks/week06-data-factory.md reports/week06
```

期望看到什么：

能看到至少这些文件或目录：

- week06-data-factory-blueprint.md
- week06-asset-graph.md
- week06-partition-backfill-strategy.md
- week06-run-evidence-spec.md
- week06_run_evidence.schema.json
- runbooks/week06-data-factory.md

怎么看结果：

`python -m json.tool` 能正常打印 schema，说明 JSON 结构合法；`ls` 能看到蓝图和报告目录，说明后续 evidence 有落点。

常见错误：

- 本机能 `ls`，devbox 不能 `ls`：compose mount 或工作目录不一致。
- schema JSON 解析失败：先修 schema 文件，不要继续生成 evidence。

下一步进入条件：

foundation 文件齐全，并且 schema JSON 能被 devbox 解析。

Step 2 | 验证 Definitions 可加载

为什么做这一步：

Definitions 可加载只说明 Dagster 能识别这些 asset、resource、check 和 job。它不等于数据已经正确写入，也不等于下游可消费，但它是所有 UI / CLI 演示的前置门槛。

执行命令：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    pytest tests/integration/test_week06_definitions_loadable.py -q
```

期望看到什么：

测试通过，输出中没有 import error、resource missing、asset key conflict。

怎么看结果：

通过后，你应该能在代码或 UI 中看到类似这些 Week06 asset 命名空间：

- week06/source/seed_manifests
- week06/factory/manifest_gate
- week06/ingestion/raw_ticket_events_partitioned
- week06/silver/ticket_fact_partitioned
- week06/evidence/run_evidence

常见错误：

- `ModuleNotFoundError`: devbox 依赖或 Python path 不对。
- asset key 重复：检查是否遗漏 `week06/*` namespace。
- resource 初始化失败：优先查 env 和 compose 服务，不要直接删 asset。

下一步进入条件：

`test_week06_definitions_loadable.py` 通过，并且你能用一句话解释：Definitions loadable 是“可编排边界通过”，不是“业务结果通过”。

Step 3 | 观察 source / manifest，不要把观察误认为资产生成

为什么做这一步：

source / manifest observation 是把外部输入、清单和批次边界带入数据工厂。它的价值是建立“输入事实”，不是直接生成下游表。

执行命令：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    pytest tests/integration/test_week06_asset_graph_smoke.py -q
```

期望看到什么：

smoke test 通过，并且能确认 Week06 最小 asset graph 可以被描述。

怎么看结果：

记录这 5 个字段，后面写 summary 时要用：

字段	要记录什么
<code>manifest_path</code>	seed manifest 或 manifest gate 使用的清单路径
<code>source_files</code>	本次分区引用了哪些输入
<code>batch_id</code>	本次输入批次如何识别
<code>expected_row_count</code>	dry-run 预期输入行数
<code>data_release_id</code>	如果存在，记录数据发布边界；没有就写 <code>not_available</code>

常见错误：

- 只看到了 source 文件，却没有记录 manifest 绑定：后续无法解释这批数据从哪里来。
- 把 observation 当成 materialization 成功：这是两件事，source 可见不代表目标 asset 已写入。

下一步进入条件：

你能说明本次分区的 source / manifest / batch 边界，并知道这些只是“输入可观测”，不是“下游可消费”。

Step 4 | Materialize 一个 daily partition

为什么做这一步：

Week06 的资产化数据工厂以 daily partition 为最小运营单位。实验默认分区是 2026-04-17，后续 backfill、checks、evidence 都围绕这个分区说话。

执行命令：

如果使用 Dagster UI，在 <http://localhost:3000> 选择 Week06 相关 asset，并使用 partition 2026-04-17 运行 materialize。

如果先走命令行 smoke，执行：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    pytest tests/integration/test_week06_asset_graph_smoke.py -q
```

期望看到什么：

- UI 里对应 asset run 启动并结束。
- 或 smoke test 通过，证明最小 asset graph 可以被运行路径触达。
- 后续报告目录中可以生成 evidence 或临时测试输出。

怎么看结果：

记录：

项	记录方式
partition_key	固定写 2026-04-17
target assets	写本次跑了哪些 Week06 assets
started / finished	从 run event 或报告里记录
output count	没有真实写入时写 not_available，不要猜
report path	指向真实存在的报告或 evidence

常见错误：

- partition key 写成日期范围：本实验的最小单位是 daily partition。
- UI 成功但报告没有：需要确认 run evidence 生成测试或报告路径。

下一步进入条件：

你已经明确本次实验围绕 2026-04-17，并能定位运行输出或说明为什么当前是 dry-run / smoke。

Step 5 | 安全制造缺口

为什么做这一步：

恢复实验要有缺口，但不能靠删除主表或破坏真实状态来制造事故。这里的目标是让 backfill / replay 决策可解释，而不是把环境弄坏。

执行命令：

优先使用 dry-run 方式生成缺口和计划，不直接删除主表：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.data_factory.backfill_plan --partition 2026-04-17 --mode dry-run
```

期望看到什么：

生成 reports/week06/backfill/backfill_plan_2026-04-17.json。

怎么看结果：

重点看 gap_reason，常见值包括：

- no_valid_input_rows
- partition_has_no_seed_rows
- dry_run_no_db_write
- week04_lakehouse_not_available
- week05_analytics_not_available

常见错误：

- 为了演示缺口直接删除主表：这是不合格实验做法。
- expected input 为 0 就修改业务代码：先查 seed 数据、manifest、partition 日期是否一致。

下一步进入条件：

你能写清楚缺口来自哪里，并确认没有破坏主表或正式输出。

Step 6 | 生成并解读 backfill dry-run plan

为什么做这一步：

dry-run plan 是操作员做恢复决策的依据。它应该告诉你补哪一天、预期输入多少、当前输出多少、为什么有缺口、怎么避免重复 side effect、下游会受什么影响。

执行命令：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    python -m pipelines.data_factory.backfill_plan --partition 2026-04-17 --mode dry-run
```

期望看到什么：

控制台输出 backfill plan，并写入：

```
reports/week06/backfill/backfill_plan_2026-04-17.json
```


怎么看结果：

字段	怎么读
<code>partition_key</code>	本次恢复对象，实验固定为 <code>2026-04-17</code>
<code>expected_input_count</code>	manifest / seed 层预期输入量
<code>current_output_count</code>	当前目标输出量；dry-run 下可能为 0 或 not available
<code>gap_reason</code>	为什么需要 retry / replay / backfill
<code>idempotency_guard</code>	如何避免重复写入或重复 side effect
<code>downstream_impact</code>	对 Week07 / Week08 / Week11 等下游的影响

常见错误：

- 只看 `proposed_action`，不看 `idempotency_guard`：会导致“补数成功但重复写入”。
- 把 dry-run 当成已经修复：dry-run 只是决策证据，不是正式 materialize。

下一步进入条件：

你能基于 plan 说明：这次应该 retry、replay、backfill、hold，还是只保留 dry-run 证据。

Step 7 | 运行 5 类 asset checks

为什么做这一步：

asset checks 是数据工厂的质量闸门。它不是为了把所有状态都写成 `passed`，而是让每个状态都有明确 reason 和后续动作。

执行命令：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run
--rm devbox \
    pytest tests/integration/test_week06_asset_checks.py -q
```

期望看到什么：

测试通过，并生成或验证 Week06 checks 输出。

怎么看结果：

Check	目标	失败或非通过时怎么处理
<code>manifest_consistency</code>	manifest 与实际输入一致	hold downstream, 先修 manifest / source 绑定
<code>row_count_output_count</code>	输入 / 输出行数合理	进入 manual review, 查 partition 与过滤条件
<code>duplicate_idempotency</code>	没有重复 side effect	禁止直接重跑, 先确认 idempotency key
<code>required_field_null_rate</code>	关键字段缺失率可控	记录 warning / failed, 并说明字段影响

Check	目标	失败或非通过时怎么处理
<code>partition_completeness</code>	该分区完整	生成 backfill / replay 决策

常见错误：

- optional Week04 / Week05 没启用却写 `passed`：必须写 `skipped` 或 `not_available`。
- `warning` 没有 reason code：warning 没有解释就无法交接。

下一步进入条件：

5 类 checks 都有结果；非 `passed` 状态有 reason code 和 downstream decision。

Step 8 | 生成 run evidence 并校验 schema

为什么做这一步：

run evidence 是 Week06 给下游的正式交接物。它必须能被 schema 校验，并且能清楚表达：发生了什么、状态是什么、证据在哪里、下游该怎么处理。

执行命令：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    pytest tests/contract/test_week06_run_evidence_schema.py tests/integration/
test_week06_run_evidence_generation.py -q
```

期望看到什么：

schema contract 和 evidence generation 测试通过，并出现类似路径：

```
reports/week06/run_evidence/*.json
reports/week06/run_evidence_summary.md
```

怎么看结果：

run evidence 至少要能解释：

字段	要求
<code>asset_key</code>	指向真实 Week06 asset
<code>partition_key</code>	固定为本次实验分区
<code>status</code>	<code>success</code> / <code>warning</code> / <code>skipped</code> / <code>not_available</code> / <code>failed</code>
<code>reason_codes</code>	数组，不是字符串
<code>report_path</code>	指向真实存在的报告
<code>downstream_decision</code>	<code>proceed_to_week07</code> / <code>manual_review_required</code> / <code>hold_downstream</code> / <code>dry_run_only</code>

常见错误：

- `reason_codes` 写成字符串。
- `report_path` 指向不存在文件。

- optional 依赖没有启用却写成 `passed`。

下一步进入条件：

run evidence schema 测试通过，且你能用 evidence 说明下游能不能继续。

Step 9 | 按 runbook 完成交接记录

为什么做这一步：

实验最后不是“跑完截图”，而是让另一个人能接手。你的 summary 要把 partition、asset、checks、evidence、downstream decision 和 known limitation 说清楚。

执行命令：

打开并补充：

```
reports/week06/week06_lab_execution_summary.md
```

建议至少写入这段交接句：

```
For partition <partition_key>, asset <asset_key> is <status>.  
Checks: <summary>.  
Evidence: <path>.  
Downstream decision: <decision>.  
Known limitation: <limitation>.
```

期望看到什么：

summary 能回答：

- 本次选择哪个 partition?
- 为什么需要 retry / replay / backfill / dry-run?
- 哪些 checks 通过、warning、skipped 或 failed?
- evidence report 路径是什么?
- 下游是否可以 unblock?
- 哪些能力仍是 optional / placeholder?

怎么看结果：

找一个没有参与实验的人，只看 summary 和 evidence path，应该能判断下一步该做什么。

常见错误：

- 只贴命令输出，没有操作结论。
- 把 `skipped/not_available` 写成失败，但没有说明它是 optional dependency。
- 没有写 downstream decision。

下一步进入条件：

`week06_lab_execution_summary.md` 是 handoff-ready，而不是个人笔记。

! 本实验不是为了截图

截图只能说明你打开过页面。实验交付要能说明：选了哪个 partition、为什么补、怎么验、证据在哪里、下游是否能继续消费。

验收清单

- ☐ `Definitions` 可加载, 且知道它只代表编排边界通过。
- ☐ 能描述 Week06 最小 asset graph。
- ☐ 默认分区 2026-04-17 有 dry-run plan。
- ☐ `partition_key`、`expected_input_count`、`current_output_count`、`gap_reason`、`idempotency_guard`、`downstream_impact` 都被解读。
- ☐ 5 类 asset checks 都有结果和 reason。
- ☐ run evidence schema / generation 测试通过。
- ☐ `report_path` 指向真实存在文件。
- ☐ optional Week04 / Week05 依赖写成 `skipped/not_available`, 没有 fake passed。
- ☐ `week06_lab_execution_summary.md` 能让另一个人接手。

常见错误与排查

问题	典型表现	排查顺序
Dagster UI unavailable	<code>http://localhost:3000</code> 打不开	先 <code>docker compose ... ps</code> , 再看端口冲突和服务日志
<code>Definitions</code> loadable fails	import error / resource missing	查 devbox 依赖、Python path、compose mount、env
<code>expected_input_count = 0</code>	dry-run plan 没有输入	查 seed 数据、manifest path、partition 日期是否为 2026-04-17
Check fails	pytest 失败 或 check status failed	先读 check reason code, 再决定 retry / replay / hold
Evidence schema fails	JSON schema contract 不通过	查 required 字段、enum 值、 <code>reason_codes</code> 类型、 <code>report_path</code>
Optional deps missing	Week04 / Week05 相关字段没有真实结果	写 <code>skipped/not_available</code> 和 reason code, 不要写 fake passed

延伸阅读

- [Dagster Docs: Partitions and Backfills](#)¹
- [Dagster Docs: Asset Checks](#)²
- [Dagster Docs: Asset Observations](#)³
- [OpenLineage: Object Model](#)⁴
- 项目 runbook: `runbooks/week06-data-factory.md`
- 项目同步说明: `reports/week06/course_site_sync_notes.md`

¹Dagster 官方文档中关于 partitions 与 backfills 的说明, 用于理解为什么 Week06 以 daily partition 作为恢复单位。

²Dagster 官方文档中关于 asset checks 的说明, 用于理解 checks 为什么是资产质量闸门, 而不是普通测试日志。

³Dagster 官方文档中关于 asset observations 的说明, 用于区分 source / manifest observation 与 materialization。

⁴OpenLineage object model 用于理解跨系统 lineage / run / job / dataset 的证据组织方式。